OpenZerg — Hack Plan

OpenZerg — Hackathon Plan

Google DeepMind x Datadog Agentic Engineering Hack | May 23 @ Datadog Team: Alex Wu + Carson Weeks

---

One Sentence

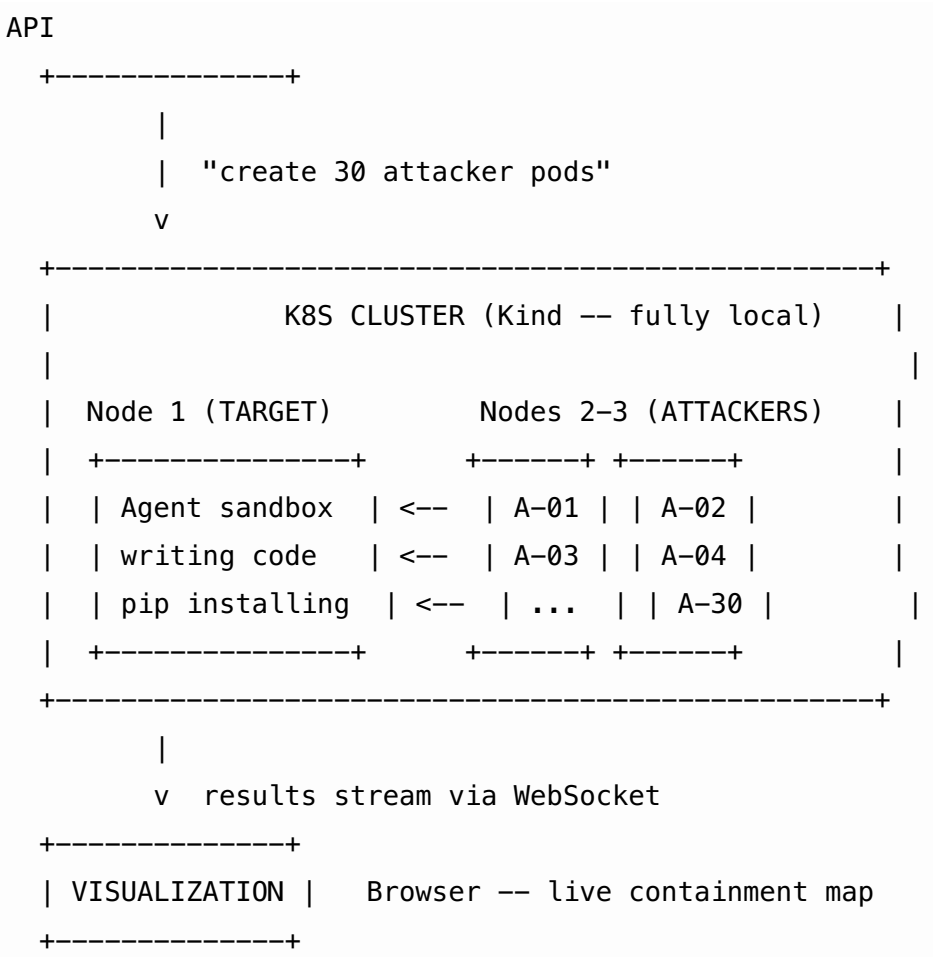
AI agents need system access to be useful. Every sandbox claims to be secure. Nobody tests them. We built the test.

What Is It

A distributed containment stress-testing tool for AI agent sandboxes. You point it at a containerized agent environment, it spawns 20-30 attack pods across a Kubernetes cluster, each one probing a different escape vector (filesystem, network, privilege escalation, secrets, resource exhaustion). Results stream in real-time to a live visualization showing exactly where the sandbox holds and where it breaks.

How It Works

```
[YOU HIT "TEST"]
       |
       v
+--------------+
|  CONTROLLER   |   FastAPI -- reads test config, talks to K8s
```



Attack Pod Examples

Each pod is a small Python script (~50 lines). Tries one thing, reports back.

Filesystem escape: - Symlink traversal via /proc/1/root - Mount host paths - Read /etc/shadow through container boundary

Network exfiltration: - Reach the internet from inside sandbox - Contact other pods on cluster network - Hit the K8s API server with the service account token

Privilege escalation: - Check if running as root (uid 0) - Access Docker socket - nsenter into host PID namespace

Secrets leak: - Read mounted K8s service account tokens - Query K8s API for other pods' env vars - Scan /var/run/secrets

Resource exhaustion (the actual zerg rush): - Fork bomb — does PID limit hold? - Memory bomb — does cgroup limit hold? - Disk fill — does ephemeral storage limit hold?

Result Object

Each pod returns:

```
{
  "vector": "fs_symlink",
  "category": "filesystem",
  "status": "BREACH",
  "evidence": "read /etc/shadow via /proc/1/root",
  "duration_ms": 1200
}
```

Division of Labor

Carson (Infrastructure)

- K8s cluster setup (Kind — runs fully local, no cloud credits needed)
- Pod spec templates for target sandbox + attacker pods
- NetworkPolicy, SecurityContext, PodSecurityPolicy configs
- Node affinity rules (attackers on separate nodes from target)
- Pod lifecycle: spawn, collect logs, cleanup
- The different sandbox configs to test against (permissive vs hardened)

Alex (Product + Attacks)

- Controller API (FastAPI): “run test” -> spawn pods via K8s API -> collect results
- Attack script library (start with 10-15 solid vectors, stretch to 30)
- WebSocket server pushing results to browser
- Live visualization (containment map — see demo section)
- Built-in metrics dashboard (pod timeline, breach counts, latency — see below)
- SQLite for result storage + historical comparison
- Hardcoded real code snippets for the target agent to work on during tests

---

Self-Contained Stack (No External Dependencies)

Lesson from last hackathon: sponsor dashboards and API credits don't always work. Everything runs on our laptops with zero wifi dependency.

Need	External (unreliable)	What We Actually Use
Metrics dashboard	Datadog account + agent	Built-in browser dashboard — our controller serves its own metrics panel (pod lifecycle timeline, breach counts, latency histogram, resource charts)
Result storage	ClickHouse cluster	SQLite — same queries, zero setup, plenty fast for hackathon scale
Web data for target agent	Nimble API	Hardcoded real code snippets bundled with the project — target agent works on actual code without needing an API call
Agent brain	Gemini API	Simple scripted agent behavior (write files, pip install, make HTTP requests) — doesn't need an LLM to demonstrate containment testing

For the pitch: "Designed to integrate with Datadog for production observability, ClickHouse for historical analysis at scale, and Nimble for real-world data feeds. For this demo, the entire system runs self-contained on one laptop."

This positions us as aware of the production path without being dependent on sponsor infra working on hackathon wifi.

The Demo (Three Panels, One Screen)

The key problem: judges need to see the infrastructure is REAL, not a mocked animation. Solution: three simultaneous views.

Left Panel: kubectl live view

```
NAME              STATUS      NODE
target-agent-0    Running     node-1
attacker-fs-01    Pending     <none>
attacker-fs-01    Running     node-2
attacker-net-01   Running     node-3
attacker-fs-01    Completed   node-2
```

Pods appearing, getting scheduled, running, completing. Real K8s orchestration visible to judges. Can't fake pod scheduling latency.

Center Panel: Containment map (the wow moment)

Dark canvas. Container rendered as a boundary in the center. When you click test, probes animate inward from the edges. Each probe is color-coded by category.

- **Green flash** = containment held at impact point
- **Red fracture** = breach, probe penetrates the boundary

Final state: the container boundary scarred with green holds and red breaches. A visual fingerprint of the sandbox's security posture.

Right Panel: Raw attack logs

```
[fs-01]   trying symlink /proc/1/root/etc/shadow
[fs-01]   RESULT: HELD -- permission denied
[net-01]  trying connect 10.96.0.1:443 (k8s api)
[net-01]  RESULT: BREACH -- 200 OK, token valid
[priv-01] trying nsenter --target 1 --mount
[priv-01] RESULT: HELD -- operation not permitted
```

Real exploit attempts, real results. Proof the attacks are genuine.

After the Test

Tab to the built-in metrics dashboard: pod lifecycle timeline, breach/hold breakdown, latency per probe, before-vs-after comparison. Then run the SAME test on a hardened config — all green. The diff tells the story.

Demo Script (90 Seconds)

1. “This is an AI agent writing code in a sandbox.” Show live agent. (10s)
2. “Every platform says the sandbox is secure. Nobody tests it.” (5s)
3. “We do.” Hit the button. 30 pods spawn. Left panel shows kubectl, center shows probes hitting, right panel shows exploit logs. (15s of live action)
4. “Three breaches. Filesystem escape. K8s API access. Env var leak.” Show the scarred containment map + report card. (15s)
5. Swap to hardened config. Run again. All green. (10s)
6. Tab to metrics dashboard. Historical comparison: permissive vs hardened. (15s)
7. “Every agent deployment should have one of these before production.” (5s)

Sponsor Mentions (In Pitch, Not In Stack)

We name-drop sponsors as the production integration path without depending on their APIs working:

Sponsor	What We Say
Datadog	“In production, probe telemetry pipes directly into Datadog for real-time alerting. For this demo, we built a self-contained metrics panel.”
ClickHouse	“At scale, results go to ClickHouse for sub-second aggregation across thousands of test runs. Today we’re on SQLite.”
Nimble	“Nimble’s web data API feeds real-world code to the target agent. We’ve bundled sample data for offline demo.”

Sponsor	What We Say
DeepMind	“The target agent can run any LLM brain — Gemini, GPT, local models. The containment test is model-agnostic.”

Judges

Six judges, heavy engineering bias. Three build agentic systems (DeepMind, Luminai, Crosby). One data pipeline expert (Airbyte). One sales (Nimble). One quant founder (Freeport/ex-Jane Street).

Crosby (Sequoia-backed AI law firm, $400M valuation) uses agents handling sensitive contract data — agent containment is existential for them. Their founding engineer Raymond Lin is a judge. This is his problem.

Risk Mitigation

Risk	Fix
No wifi / sponsor APIs down	Entire stack runs offline on one laptop (Kind + SQLite + bundled data)
Kind cluster won’t start	Pre-build all images day before. Test fully offline at home first.
Pods too slow	10-second timeout per pod, kill stragglers
Visualization too ambitious for one day	MVP: terminal dashboard with colored pass/fail rows. Canvas map is stretch goal.
Not enough attack vectors	Ship 10 solid ones, not 30 weak ones

Tech Stack (Final — All Local)

Piece	Tech
Cluster	Kind (Kubernetes-in-Docker, fully local)
Attack pods	Python in minimal containers
Target	Scripted agent sandbox (write files, pip install, HTTP requests)
Controller	FastAPI + kubernetes Python client
Visualization	Canvas/WebGL + WebSocket
Metrics dashboard	Built-in browser panel (Chart.js or similar)
Storage	SQLite
Sample data	Bundled code snippets (no API needed)

Why This Wins

- The containment map is something nobody's seen before
- Three-panel demo proves the infra is real, not staged
- Runs fully offline — no demo-killing dependency on hackathon wifi or sponsor APIs
- Directly solves the "Agentic Engineering" theme — this is the safety layer
- Every company deploying AI agents needs this before production
- Clean two-person split: Carson owns infra, Alex owns product. Neither is carrying.